

Python 学习知识点速记版

核心考点 · 易错点 · 高频命令（函数 → 文件 → 模块 → 面向对象 → 网络 → 并发 → MySQL → Redis）

一、函数

1.1 作用域与关键字

- 查找顺序 LEGB: Local → Enclosing → Global → Built-in, 逐层向外。
- global: 函数内修改全局变量须先声明; nonlocal: 修改外层（非全局）函数的局部变量。
- 函数内对变量赋值即视为局部, 未声明 global 直接改全局会报 UnboundLocalError。

1.2 返回值与 lambda

- return 结束函数并返回; 无 return 返回 None; 可一次返回多个值（打包成元组）。
- lambda 参数: 表达式, 只能写单行表达式。

1.3 高阶函数与内置

- 函数是一等公民, 可作参数或返回值。filter(条件,序列) 过滤、map(func,序列) 映射、sorted(iter,key=,reverse=) 排序。
- 易漏: str() 面向人、repr() 面向解释器; == 比内容（值）、is 比身份（内存地址）; id() 取地址。

```
x = 1
def f():
    global x
    x += 1    # 修改全局须先声明 global
```

二、字符编码与文件操作

2.1 常用模式

模式	作用
r	只读（默认，文本）
rb	二进制只读（图片/视频）
w	覆盖写入（文本，清空原内容）
wb	二进制写入
a	追加写入
r+ / w+	读写

- 编码演进：ASCII → GBK → Unicode → UTF-8；`str.encode()` 编码为 bytes，`bytes.decode()` 解码回 str。

2.2 关键写法

- `with open(...) as f`：自动关闭资源，推荐使用。
- `read` 全读、`readline` 读一行、`readlines` 读所有行为列表、`for` 遍历逐行。
- 爬虫下载：`res.content` 是二进制，必须用 `wb` 模式写。

```
with open("a.jpg", "wb") as f:  
    f.write(res.content)    # 图片/视频用 wb 二进制写
```

三、常用模块

模块	常用	用途
random	random/randint/choice/sample	生成随机数
time	time()/sleep()/localtime/strftime	计时与延迟
datetime	datetime/date/timedelta/ strftime/strptime	日期计算
sys	argv/exit/path/version	与解释器交互
os	getcwd/mkdir/remove/rename/ listdir + os.path	操作系统操作
json	dumps/loads/dump/load	序列化（传输/持久化）
loguru	logger.info(...)	简单日志

```
import json  
s = json.dumps(d)    # 字典 → 字符串  
d2 = json.loads(s)   # 字符串 → 字典
```

四、模块与包

- 导入：import 模块 / from 模块 import 名 / import 模块 as 别名；* 导入全部（慎用）。
- `__name__`：直接运行时为 `__main__`，被导入时为模块名；常用来写程序入口。
- 循环导入：A 与 B 顶层互相 import 会报错，应避免在模块顶层相互导入。
- 包 = 含 `__init__.py` 的目录；导入本质是按 `sys.path` 查找，绝对导入常用 `BASE_DIR`。
- 虚拟环境：python -m venv venv 创建；Linux 用 `source venv/bin/activate`、Windows 用 `venv\Scripts\activate`；`pip freeze > requirements.txt` 导出依赖。

五、面向对象基础

- 类=模板、对象=实例；实例方法首参 `self` 指向实例本身。
- `__init__` 构造方法：实例化时自动调用，初始化实例属性。实例化三步骤：创建空对象 → 调用 `__init__` → 返回对象。
- 类属性所有实例共享；类方法 `@classmethod` 首参 `cls`；静态方法 `@staticmethod` 无 `self/cls`。

```
class Circle:
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r ** 2
```

六、面向对象进阶

- 继承：子类复用父类，可重写同名方法；`super().__init__()` 调用父类构造。
- 封装：`__foo` 双下划线触发名称改写 `_类名__foo`（非真正权限）；`property` 把方法转成属性访问。

- 多态/鸭子类型：只要对象有对应方法即可当作目标类型使用；isinstance 能识别继承链、type 只返回精确类型。
- 反射：getattr / setattr / setattr / setattr 按字符串名动态访问属性方法。
- 魔法方法：__str__ / __eq__ / __len__ / __getitem__ 等。
- 异常：try / except / else / finally；raise 主动抛；自定义异常需继承 Exception。

```
if not hasattr(obj, "name"):
    raise ValueError("缺少属性")    # 主动抛异常
```

七、网络编程

要素	作用
IP 地址	定位设备
端口	定位进程（0-65535，80 为 HTTP）
协议	保证沟通（TCP/UDP）

- 架构：CS 需安装客户端（一对多）；BS 浏览器即客户端、基于 HTTP，是 Web 基础。
- TCP（SOCK_STREAM）可靠有序、有粘包；UDP（SOCK_DGRAM）高效不可靠、有数据边界。
- 服务端：socket → bind → listen → accept → recv/send → close；客户端：socket → connect → send/recv → close。
- send/recv 传字节串：发送前 encode()、接收后 decode()；accept/recv/input 是三大阻塞函数。
- 粘包：TCP 无边界。解法：struct.pack("i",len) 发 4 字节长度头，先 recv(4) 取长度再循环收。

```
server.bind(("127.0.0.1", 8899))
server.listen(5)
conn, addr = server.accept()
conn.send(conn.recv(1024).upper())    # 大小写转换服务
```

八、并发编程

机制	调度	资源	切换开销	适用
进程	操作系统	独立内存	毫秒级	CPU 密集
线程	操作系统	共享内存	微秒级	IO 密集
协程	用户态	单线程内数万	极小	高并发 IO

- 多道技术：空间复用（内存分片）+ 时间复用（时间片轮转 / IO 阻塞让出 CPU）；单核是并发、多核才是并行。
- threading：Thread(target=,args=) 创建、start() 置就绪、join() 等待完成。
- 线程池 ThreadPoolExecutor(max_workers)：submit 返回 Future，先全部提交再统一取 result()（过早调用会串行化）。
- 互斥锁 Lock：acquire/release 成对，临界区串行；函数内改全局变量需 global。
- queue.Queue：put/get 默认阻塞、线程安全，用于生产者-消费者模型解耦。
- GIL：进程内全局锁，Python 多线程无法并行；multiprocessing 每个进程独立 GIL 可并行，须放 if __name__=="__main__"。
- asyncio：async def 定义协程、await 等待（用 asyncio.sleep() 不用 time.sleep()）、asyncio.run() 入口、create_task + gather 收集结果。

```
import threading
t = threading.Thread(target=work, args=(3,))
t.start(); t.join()    # 先启动，再等待完成
```

九、MySQL 基础知识

- 层级：数据库 → 表 → 记录（行）→ 字段（列）。
- SQL 分类：DDL 建删表 / DML 增删改 / DQL 查询 / DCL 权限事务。
- 增删改查：INSERT 插入、UPDATE 更新（务必带 WHERE）、DELETE 删除（务必带 WHERE）、SELECT 查询。
- 查询：WHERE 过滤、ORDER BY 排序、LIMIT 分页；聚合 COUNT/SUM/AVG 配 GROUP BY、HAVING 过滤分组、JOIN 联表。

- 约束：主键 PRIMARY KEY、NOT NULL、UNIQUE、外键 FOREIGN KEY；索引引用 B+ 树加速查询。
- PyMySQL：connect → cursor → execute → fetchall → commit → close（增删改后必须 commit）。
- 事务 ACID：原子性/一致性/隔离性/持久性；try 中全部成功 commit，except 中 rollback。

```
cur.execute("SELECT * FROM student WHERE age > 18")
rows = cur.fetchall()
conn.commit()    # 增删改后必须提交
```

十、Redis 基础知识

Redis 是基于内存的高性能键值数据库，常用作 MySQL 之上的缓存层，读写极快。

结构	特点	典型用途
string	键值对	缓存、计数
hash	字段-值映射	存对象（用户/商品）
list	双向链表	消息队列
set	无序不重复	去重、共同好友
zset	带分数有序	排行榜

- 常用命令：SET/GET/DEL、HSET/HGET、LPUSH/RPOP、SADD/SMEMBERS、ZADD/ZRANGE。
- EXPIRE 设过期时间、TTL 查剩余；持久化 RDB（快照）+ AOF（日志）。
- redis-py：redis.Redis(host,port,decode_responses=True)，set/get/hset/hgetall。
- 场景：热点缓存（防穿透/击穿/雪崩）、消息队列、计数器/排行榜、分布式会话。

```
r = redis.Redis(host="localhost", port=6379, decode_responses=True)
r.set("name", "zhang")
print(r.get("name"))    # b/zhang
```